

Versal I2C C++ implementation lib

some examples

Pelayo Leguina

Protocolo I²C

Características Principales

Comunicación serie síncrona entre dispositivos en un mismo PCB.

Utiliza **dos líneas**:

- **SDA (Serial Data)** – Línea de datos.
- **SCL (Serial Clock)** – Línea de reloj.

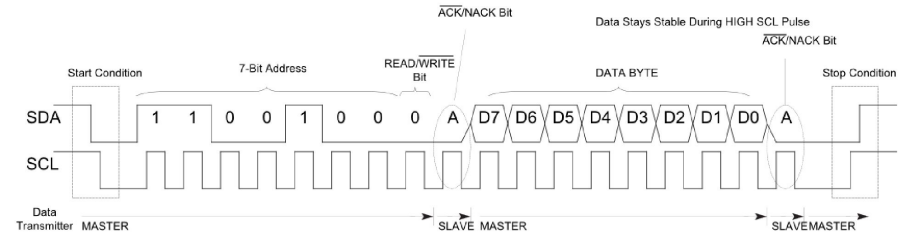
Soporta múltiples **maestros y esclavos** en un mismo bus.
Dirección de dispositivos de **7 o 10 bits**.

Velocidades estándar:

- **100 kHz (Estándar)**
- **400 kHz (Rápido)**
- **1 MHz (Alta velocidad)**
- **3.4 MHz (Ultra rápida)**

Simplicidad y Eficiencia en Hardware

- Solo necesita **dos líneas (SDA y SCL)** en comparación con otros protocolos como **SPI** que requieren más señales.
- Permite reducir el número de **pines de microcontroladores** y FPGA, ahorrando espacio y costos en el diseño del PCB.



Estructura de una Transmisión

1. **Condición de Inicio (START)** – El maestro baja SDA mientras SCL está alto.
2. **Dirección del esclavo + bit de lectura/escritura.**
3. **ACK/NACK** – Confirmación del esclavo.
4. **Envío/Recepción de datos (8 bits por transferencia).**
5. **ACK/NACK tras cada byte.**
6. **Condición de Parada (STOP)** – El maestro sube SDA mientras SCL está alto.

Entender la arquitectura del PMC y CIPS en Versal

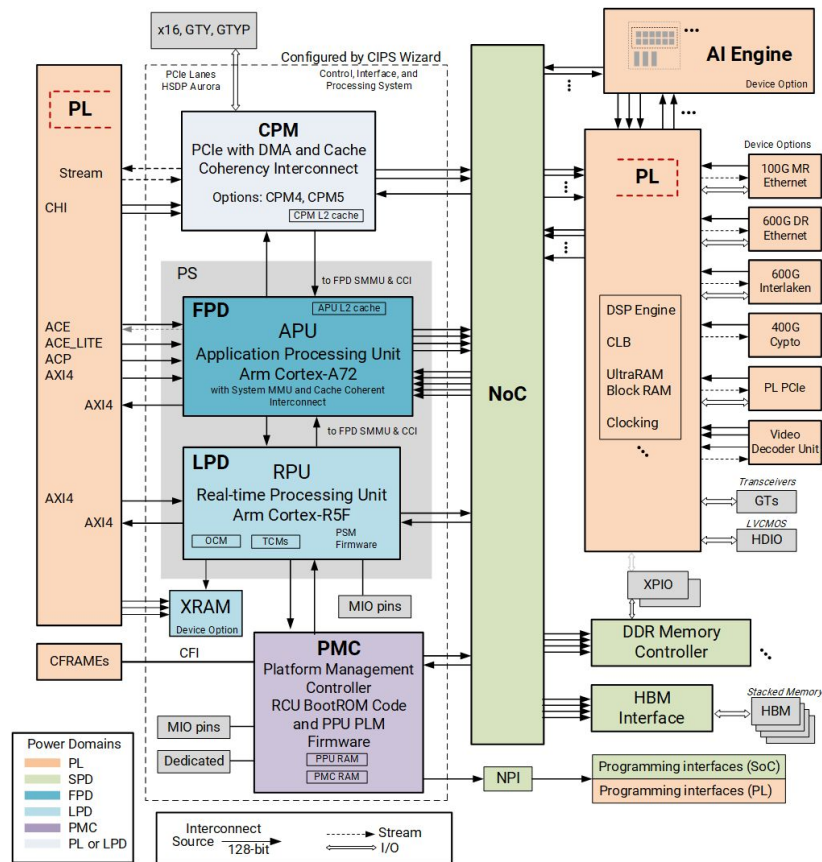
En la arquitectura Versal, todos los periféricos están bajo el control inicial del PMC, que actúa como intermediario entre el PS (Processing System, que incluye la APU) y los periféricos externos.

📌 ¿Qué hace el PMC en Versal?

- ✓ Inicializa el hardware antes de arrancar la APU (PS)
- ✓ Gestiona el acceso a periféricos como I²C, SPI, UART, GPIO
- ✓ Controla la seguridad y permisos de acceso a los periféricos
- ✓ Administra la carga de bitstreams en el PL (FPGA)

♦ Diferencia con Zynq MPSoC:

En Zynq MPSoC (UltraScale+), el PS tenía acceso directo a los periféricos. **En Versal, todo pasa primero por el PMC**, y luego el PS (APU) puede tomar el control.





El **CIPS (Configurable Intelligent Processing System)** es un bloque IP de **Vivado** que define cómo se configurará el **Processing System (PS)** en la Versal.

📌 ¿Qué hace el CIPS?

- ✓ Define la configuración de los **procesadores (APU, RPU, PMC)**
- ✓ Controla el acceso a los **periféricos como I²C, UART, SPI, GPIO, CAN, USB, PCIe, Ethernet**
- ✓ Maneja la inicialización del sistema y la seguridad
- ✓ Administra la **conexión entre el PS (procesador) y el PL (FPGA fabric)**
- ✓ Permite seleccionar si la Versal funcionará en **modo baremetal** o con un sistema operativo como **Linux**

📌 ¿Qué componentes forman parte del CIPS?

El **CIPS** está dividido en tres secciones principales:

1 APU (Application Processing Unit)

- Contiene hasta **cuatro núcleos ARM Cortex-A72** (para Linux/Baremetal).
- Incluye cachés L1/L2 y un MMU para administrar la memoria.

2 RPU (Real-Time Processing Unit)

- Contiene dos núcleos **ARM Cortex-R5F** para procesamiento en tiempo real.
- Usado para tareas determinísticas y de bajo consumo.

3 PMC (Platform Management Controller)

- Es el **primer procesador que se ejecuta al encender la Versal**.
- Maneja la inicialización de hardware y seguridad.
- Gestiona el acceso a periféricos **I²C, UART, GPIO, SPI** antes de entregarlos a la APU.

♦ **IMPORTANTE:** Todo el hardware en Versal **pasa primero por el PMC** antes de que la APU o la RPU puedan usarlo.

Configuración del PMC y CIPS en Vivado

Antes de programar la APU, hay que **configurar correctamente el CIPS y definir cómo la APU accederá a los periféricos**.

Pasos en Vivado para configurar el PMC y CIPS

1 Abrir Vivado y crear un nuevo Block Design (BD)

- Inicia un nuevo proyecto y abre **IP Integrator**.
- Agrega el bloque **Versal CIPS** desde la ventana de IP.

2 Configurar el CIPS para la APU y los periféricos

- Abre la configuración del **CIPS** (doble clic en el bloque).
- En la pestaña **PS Configuration**, habilita:
 - **APU (Application Processing Unit)** → Para ejecutar Linux/Baremetal.
 - **I²C Controller 0/1** → Para sensores, PMBus, etc.
 - **UART 0/1** → Para depuración con consola.
 - **SPI / GPIO** si los necesitas para otros dispositivos.

3 Habilitar las conexiones entre el PMC y la APU

- En la configuración del CIPS, verifica que el **PMC no bloquee el acceso a I²C, UART o SPI**.
- **Habilita los permisos** para que la APU pueda acceder directamente a estos periféricos.

4 Conectar el CIPS al PL (si es necesario)

- Si algún periférico está en la FPGA (PL), añade un **AXI I²C Controller** o usa **PL-to-PS AXI Interconnect** para comunicarlo con la APU.

5 Configurar los relojes y resets

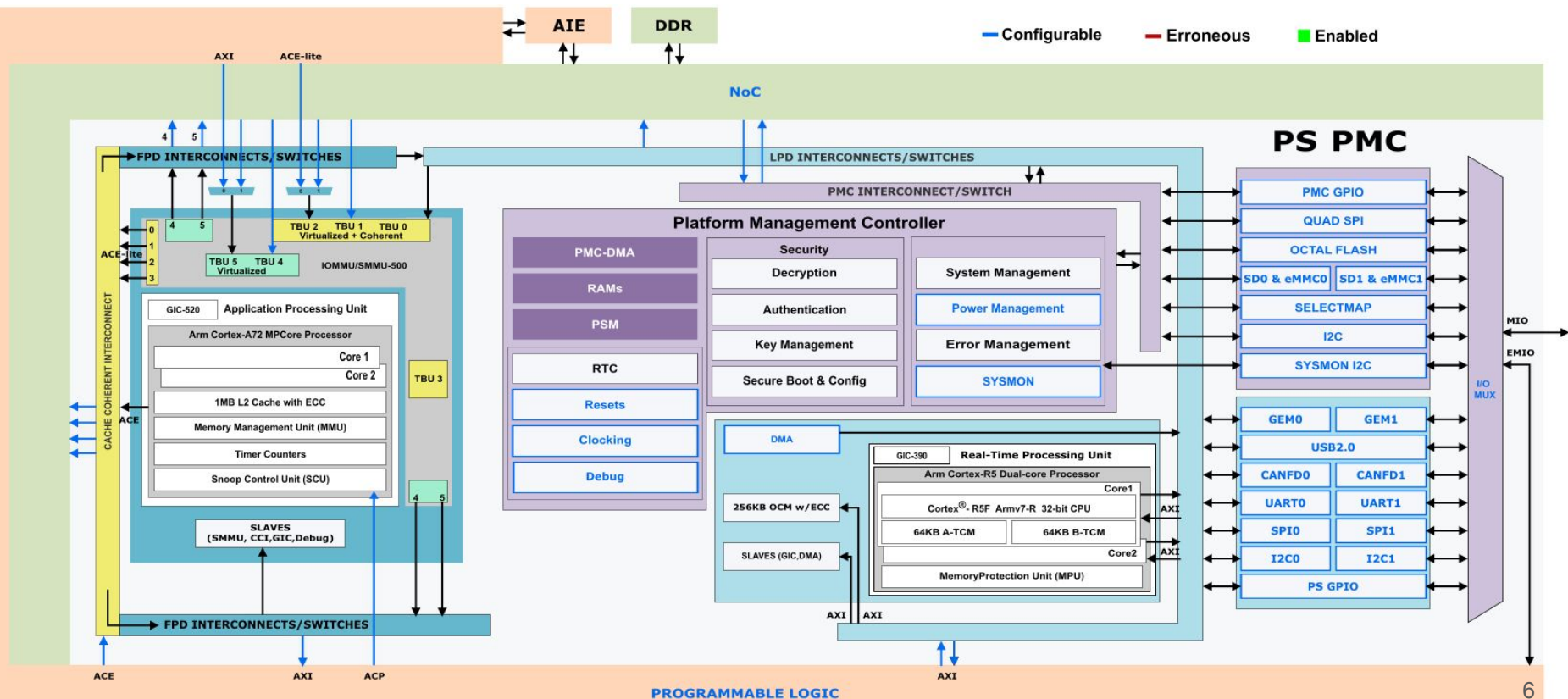
- Usa **100 MHz** como frecuencia base.
- Conecta los resets de los periféricos correctamente.

6 Validar el diseño y generar el bitstream

- Guarda el diseño y verifica que no haya errores.
- Genera el **bitstream** y exporta el hardware en formato **XSA**.

Arquitectura interna de la tarjeta

Pantalla de configuración del CIPS (azul lo configurable)



Otra info

Por supuesto, todo esto se puede LEER

<https://docs.amd.com/r/en-US/ug1273-versal-acap-design/Overview>

Nosotros, de momento, partimos de una pre-configuración por defecto, así que no necesitamos meternos a configurar nada en vivo

La **BSP (Board Support Package)** que Xilinx proporciona para la **Versal VCK190** ya viene preconfigurada con la mayoría de los controladores y configuraciones necesarias para usar los **periféricos integrados en el hardware**. Sin embargo, hay algunos aspectos que debes verificar y personalizar dependiendo de tu aplicación.

♦ 1. ¿Qué incluye la BSP por defecto de Xilinx?

Cuando descargas y usas la **BSP oficial de la VCK190**, ya viene con:  **Configuración del CIPS** predefinida para la Versal.

-  **Drivers para periféricos** como I²C, SPI, UART, GPIO, Ethernet, PCIe.
-  **Sistema de archivos y kernel preconfigurado en Linux** (si usas Petalinux).
-  **Soporte para la APU (Cortex-A72) y la RPU (Cortex-R5F).**
-  **Interfaz PS-PL habilitada** (pero debes verificar la configuración en Vivado).

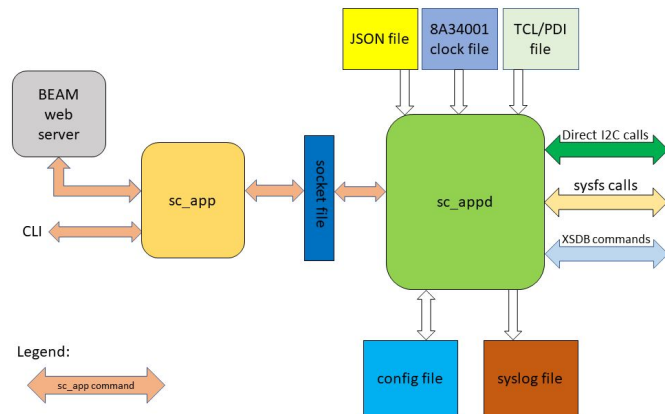
 **IMPORTANTE:** La BSP te permite usar periféricos sin necesidad de reconfigurar el CIPS en Vivado, **siempre y cuando uses la configuración estándar de Xilinx.**

System controller library

El “system controller” (SC) es una aplicación (o conjunto de aplicaciones/librerías) que se encarga de centralizar y administrar varias funcionalidades de hardware de la placa VCK190. Entre las tareas típicas se incluyen:

1. **Lectura de sensores:** Por ejemplo, sensores de voltaje, corriente y temperatura como el INA226, o la configuración de relojes (clocks) a través de chips como Si570, IDT_8A34001, etc.
2. **Gestión de periféricos I2C:** La aplicación abre y gestiona varios buses I2C (por ejemplo `/dev/i2c-0`, `/dev/i2c-1`, `/dev/i2c-2`, etc.) y los usa para leer y escribir registros de los diversos dispositivos de la placa.
3. **Control de GPIOs** y “IO Expanders” (por ejemplo, TCA6416A).
4. **Manipulación de voltajes con PMICs** (IR35215, IRPS5401, IR38164...) para cambiar o leer la configuración de voltajes del sistema.
5. **Operaciones de prueba o “Board Integration Tests” (BITs):** Validar y testear distintos subsistemas del board (por ejemplo, verificar IDCODE, voltajes, clocks, memorias DIMM, etc.).
6. **Interpretación de un archivo JSON** (VCK190.json) que describe la topología y características del hardware (sensores, buses I2C, direcciones, valores por defecto, etc.).

En un sistema real, la aplicación se compila y corre posiblemente sobre un microcontrolador o sobre un procesador principal (MPSoC / APU), y se comunica con los dispositivos de hardware mediante drivers del sistema operativo (por ejemplo, en Linux, se usan las interfaces `/dev/i2c-X` para el acceso a I2C).



System Controller Application

© Copyright 2022 AMD

<https://github.com/Xilinx/system-controller-app>

<https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/2273738753/Versal+Evaluation+Board+-+System+Controller+-+Update+7>

System controller library

SC_APP actúa como un **servidor de control** que permite acceder a periféricos a través de comandos estructurados. Su función principal es:

- ✓ Leer y escribir datos en **sensores I²C**
- ✓ Administrar dispositivos de hardware como **voltímetros, relés y EEPROMs**
- ✓ Configurar **frecuencias de reloj**
- ✓ Controlar el **estado del sistema de alimentación**

Los archivos principales que controlan la SC_APP son:

- **sc_app.c / sc_app.h** → Lógica de aplicación principal
- **sc_board.c** → Interacción con la placa y periféricos
- **sc_common.c** → Funciones comunes
- **sc_parse.c** → Procesamiento de comandos
- **sc_BIT.c** → Built-in tests (BITs)
- **sc_appd.c** → Daemon para la ejecución en segundo plano
- **VCK190.json** → Archivo de configuración con direcciones I²C

La SC_APP **no solo maneja I²C**, sino que también controla **GPIOs, voltajes, relojes y estados del sistema**.

Estructura General de sc_app

1. El programa define muchos **comandos** (e.g., **gettemp**, **getvoltage**, **getpower**...), cada uno asociado a una función principal de operaciones (por ejemplo, **Temperature_Ops**, **Power_Ops**, **Voltage_Ops**, etc.).
2. Al arrancar, **sc_app** crea un **socket UNIX** y se queda a la escucha de peticiones de un cliente (en un bucle infinito). Cada vez que recibe un mensaje:
 - Lo **parsea** en busca de la opción **-c <command>**, **-t <target>** y **-v <value>**.
 - Ejecuta la operación asociada a ese comando.
3. Buena parte de los sensores (voltajes, INA226, EEPROM, etc.) se describen en un **JSON** (por ejemplo, **VCK190.json**). El campo **Plat_Devs** (tipo **Plat_Devs_t**) contiene punteros a distintas estructuras (e.g., **INA226s**, **Voltages**, **Temperature**, etc.), cada una con info de direcciones, buses, registros, etc.

Funcionamiento de la aplicación sc_app

Veamos el flujo a grandes rasgos cuando se recibe, por ejemplo, un comando para leer un sensor de voltaje o corriente:

1. **Parse de argumentos** en `Parse_Options(int argc, char **argv)`.
2. **Identifica** qué comando se solicitó (`Command_Arg`), p. ej. `gettemp`, `getpower`, `getvoltage`.
3. **Localiza** la estructura de datos apropiada en `Plat_Devs->...` (por ejemplo, `Plat_Devs->INA226s` para leer corrientes, potencias, etc.).
4. **Invoca** la operación correspondiente, p. ej. `Temperature_Ops()` para temperatura, `Voltage_Ops()` para voltajes, `Power_Ops()` para INA226, etc.
5. Cada función `_Ops()` (por ejemplo, `Power_Ops()`) **abre el bus I²C**, posiciona la dirección del dispositivo, y ejecuta una **lectura** (o escritura) en registros concretos.

Ejemplo: `getpower -t VCCINT`

- Llama a `Power_Ops()`
- Ubica la entrada `INA226_VCCINT` en `Plat_Devs->INA226s`.
- Llama a `Get_Power(...)` que hace la lectura I²C en el INA226 (registros de bus voltage, shunt voltage, etc.).
- Devuelve la medición y `sc_app` la imprime.

Dispositivos mapeados en la app

La app no “hardcodea” todos los dispositivos. En su lugar, usa un **archivo JSON** que describe:

- **I2C_Bus** (ej. `"/dev/i2c-6"`)
- **I2C_Address** (ej. `"0x40"`)
- Sensores de tipo `"INA226_VCCINT"`, `"INA226_VCC_SOC"`, etc.
- Rango de voltajes para cada regulador
- Configuración de FMC, SFP, EEPROM, etc.

El **parse** del JSON se hace en `Parse_JSON(...)` (no se ve en este snippet completo, pero en general esa es la lógica). Llenan la estructura `Plat_Devs_t` con datos sobre direcciones, buses y funciones de cada sensor.

Al final, cuando un usuario ejecuta `getpower -t VCCINT`, el código **encuentra** en `Plat_Devs->INA226s->INA226[ ]` un sensor llamado `"VCCINT"` con su bus y dirección. Luego hace la lectura real.

Importante: Los buses i2c indicados en el JSON son buses del ZYNQ, no del apu, por lo tanto en nuestro caso no son los mismos y habría que modificarlos

Dispositivos en JSON

El JSON principal (VCK190.json) contiene un objeto "VCK190" que agrupa secciones. Cada sección (por ejemplo "CLOCK", "INA226", "VOLTAGE", "BITS", "GPIO") describe un tipo de dispositivo. Dentro de cada sección hay claves que representan los distintos dispositivos (por ejemplo "SI570_ZSFP", "INA226_VCCINT", etc.). Cada clave es un objeto que contiene la información necesaria:

- **Nombre** (p.ej. "Name" : "VCCINT").
- **Parámetros de operación** (direcciones, límites, configuraciones, buses).
- **Opcionalmente**, otros campos que varían según el tipo de dispositivo (por ejemplo "Page_Select" para PMBus).

```
{
  "VCK190" : {
    "FEATURE" : { ... },
    "BOOTMODES": { ... },
    "CLOCK": {
      "SI570_ZSFP": {
        "Name": "...",
        "I2C_Bus": "/dev/i2c-8",
        "I2C_Address": "0x5d",
        ...
      },
      "IDT_8A34001_FMC2": {
        ...
      },
      ...
    },
    "INA226": {
      "INA226_VCCINT": {
        "Name": "VCCINT",
        "I2C_Bus": "/dev/i2c-4",
        "I2C_Address": "0x40",
        ...
      },
      ...
    },
    ...
  },
  ...
}
```

```
/*
 * Board-specific Devices
 */
typedef struct {
    FeatureList_t    *FeatureList;
    BootModes_t      *BootModes;
    Clocks_t         *Clocks;
    INA226s_t        *INA226s;
    Power_Domains_t  *Power_Domains;
    Voltages_t       *Voltages;
    Temperature_t    *Temperature;
    DIMMs_t          *DIMMs;
    GPIOs_t          *GPIOs;
    GPIO_Groups_t    *GPIO_Groups;
    IO_Exp_t         *IO_Exp;
    OnBoard_EEPROM_t *OnBoard_EEPROM;
    Daughter_Card_t   *Daughter_Card;
    SFPs_t           *SFPs;
    FMCs_t           *FMCs;
    Workarounds_t    *Workarounds;
    BITS_t           *BITS;
    Constraints_t     *Constraints;
} Plat_Devs_t;
```

La aplicación sc_app, al inicializarse, seguramente llama a rutinas (por ejemplo en `sc_parse.c`) que abren este JSON, buscan las claves que necesitan y construyen estructuras en C/C++ (o la forma que sea) para que en el resto del programa estén disponibles las configuraciones de cada dispositivo.

Lectura/escritura de sensores

El archivo JSON incluye un bloque “INA226” con entradas como “INA226_VCCINT”, “INA226_VCC_SOC”, “INA226_VCCAUX”, etc. Cada una describe:

- El **bus I2C** usado, por ejemplo “I2C_Bus” : “/dev/i2c-4”.
- La **dirección I2C** del sensor, por ejemplo “I2C_Address” : “0x40”.
- Otros parámetros como la resistencia shunt (“Shunt_Resistor”: 5000) y el máximo de corriente.

La aplicación de system controller típicamente:

1. **Parsea** el JSON para obtener estos datos.
2. **Abre** el bus I2C correspondiente (por ejemplo, /dev/i2c-4).
3. **Configura** el dispositivo (INA226 en este caso) escribiendo en sus registros.
4. **Lee** periódicamente los registros de medición de voltaje y corriente.
5. Aplica las fórmulas necesarias (p.ej., usando el valor de la resistencia shunt) para convertir la lectura en amperios y vatios.

La escritura y lectura sobre I2C se hace normalmente con llamadas a funciones del tipo `ioctl()` en Linux o funciones equivalentes si es un entorno bare-metal. En el código (por ejemplo en `sc_common.c`) se podría ver una secuencia del estilo:

```
// Pseudocódigo representativo
fd = open("/dev/i2c-4", O_RDWR);
ioctl(fd, I2C_SLAVE, 0x40);
// Configuración: escribir en registros de configuración
i2c_write(fd, config_reg, valor_config);
// Lectura: leer valores de voltaje/corriente
volt_reg_val = i2c_read(fd, voltage_reg);
curr_reg_val = i2c_read(fd, current_reg);
...
```

Configuración de clocks

En la sección "CLOCK" del JSON, se ven múltiples nodos (por ejemplo "SI570_ZSFP", "IDT_8A34001_FMC2", etc.). Cada uno indica:

- El bus I2C ("I2C_Bus" : "/dev/i2c-8")
- La dirección ("I2C_Address" : "0x5d")
- Parámetros de frecuencia por defecto y límites ("Default_Freq", "Upper_Freq", "Lower_Freq", etc.)
- Posiblemente algún registro de control en el FPGA ("FPGA_Counter_Reg"), para medir la frecuencia real.

La aplicación, siguiendo la misma mecánica:

1. Parseará el JSON.
2. Accede al bus I2C y escribirá en los registros del chip de reloj para fijar la frecuencia deseada o, en su caso, para leer la frecuencia actual.
3. Si es necesario, leerá el "FPGA_Counter_Reg" (a través de un mapa de direcciones de memoria o un driver) para verificar la frecuencia real medida.

Más funcionalidades de la aplicación

Además de la gestión de sensores y relojes, se incluyen:

1. Lectura y control de GPIO.

- El JSON describe grupos de GPIO ("`GPIO_Group`", "`GPIO`") con labels como "`SYSCTLR_VERSAL_MODE0`", "`SYSCTLR_VERSAL_MODE1`", etc.
- El SC app puede así leer el modo de arranque, detectar interruptores DIP, leer interruptores (SW) o botones (PB), encender LEDs, etc.

2. Configuración y lectura de PMIC (p.ej. IR35215, IRPS5401, IR38164).

- En la sección "`VOLTAGE`" se describe cada rail: su nombre, bus I2C, dirección, y páginas PMBus.
- El SC app lee y escribe registros PMBus (protocolo basado también en I2C) para ajustar o verificar voltajes.

3. Tests o BIT ("Board Integration Tests").

- La sección "`BITs`" en el JSON define pruebas (IDCODE verify, EFUSE verify, QSPI test, DRAM mem test, etc.).
- Algunas pruebas son automáticas y otras son manuales (por ejemplo, el test de LEDs pide al usuario que observe si parpadean).
- Muchas pruebas parecen usar scripts "TCL_File" (por ejemplo: "`versal_bit_download.tcl 0x01`") con la operación "`Plat_BIT_Op`" : "`XSDB_BIT`", que sugiere que la aplicación lanza secuencias XSDB para realizar tests a nivel de Versal.

4. Soporte de tarjetas hijas (Daughter Card) como X-EBM, FMC, SFP/ QSFP, etc. para manejar EEPROMs, detectarlos, leer su presencia, y alimentaciones asociadas.

Funciones genéricas

Viendo la consistencia en la configuración, la SC app probablemente define (en archivos como `sc_common.c` y `sc_board.c`) un conjunto de funciones genéricas para:

- **Abrir bus** (por ejemplo, `i2c_open("/dev/i2c-0")`).
- **Seleccionar dirección** (por ejemplo, usando `ioctl(fd, I2C_SLAVE, dev_addr)`).
- **Escribir / leer** en registros (por ejemplo, `i2c_write_reg(fd, reg, data)` e `i2c_read_reg(fd, reg, &data)`).

Internamente, muchas de esas funciones se apoyan en la API de Linux o en drivers específicos, pero el patrón es siempre el mismo:

1. Abrir el bus (o usar un bus ya abierto).
2. Hacer `ioctl()` para establecer la dirección.
3. Preparar buffers de lectura/escritura.
4. Llamar a `read()` o `write()` en el archivo de dispositivo.

Dependiendo del dispositivo, se manejan secuencias de registros (por ejemplo, en el INA226 hay varios registros configurables) y protocolos de lectura (a veces es “read register → read data” en dos bytes, etc.).

En resumen

1. El **usuario** manda un comando al socket (por ejemplo, `-c getpower -t VCCINT`).
2. `sc_app` hace **matching** del comando con su operación (`Power_Ops`).
3. Ubica en `Plat_Devs->INA226s` la entrada "`VCCINT`" (bus, dirección).
4. **Abre** el bus I²C, **escribe** registros de calibración, y **lee** los registros de medición (bus voltage, shunt voltage, etc.).
5. Realiza las **fórmulas** finales (corriente y potencia).
6. Muestra el resultado via `SC_PRINT`.

Sí, la app se basa en macros de lectura/escritura I²C, y toda la **información de dirección** (bus, dirección, tipo de sensor) está en el JSON (`VCK190.json`) que se carga en `Plat_Devs`. De esta manera, la SC_APP "**sabe**" dónde buscar cada sensor y qué registros de I²C utilizar.

Puntos clave

- **Comandos:** Se definen en el array `Commands[ ]` (ej. `GETPOWER`, `GETTEMP` ...).
- **Flujo:** `main()` → parse → `(*Command.CmdOps)()` → la función `_Ops()`.
- **I²C:** macros `I2C_READ/WRITE`, junto con bus dev `/dev/i2c-X`.
- **Datos:** Vienen del JSON, se guardan en `Plat_Devs->INA226s`, `Voltages`, `Temperature`, etc.
- **Resultado:** Se imprime por `SC_PRINT`.

En resumen, el **acceso a sensores** en la `sc_app` se logra a través de un **sistema de comandos** que llama a funciones que **abren el bus I²C y leen/escriben registros**. Cada sensor se describe en el **JSON** y la aplicación combina esa **información de hardware** con **macros** de lectura I²C para obtener (o ajustar) los valores.

I2C

El archivo principal (`sc_app.c`) define macros y funciones para escribir/leer del bus I²C:

- **Macros:**
 - `I2C_READ(FD, Address, Len, Out, In, Ret)` → envía un byte con la dirección de registro y recibe `Len` bytes.
 - `I2C_WRITE(FD, Address, Len, Out, Ret)` → escribe `Len` bytes en la dirección I²C dada.
- **Uso de `ioctl(I2C_SLAVE_FORCE, Address)`** para fijar la dirección en el bus.
- **Flujo:** se abre el bus con `open(INA226->I2C_Bus, O_RDWR)`, luego se hace `ioctl(...)`, luego `read()` o `write()`.

En la práctica, **cada sensor** se maneja igual:

1. Abrimos `FD = open(...)`.
2. `ioctl(FD, I2C_SLAVE_FORCE, deviceAddress)`.
3. Preparamos el buffer con el registro a leer.
4. `write()` el registro (o con las macros).
5. `read()` el resultado.

Veamos qué hace cada uno:

sc_app.h

1. I2C_READ_BYTES

- Hace una operación compuesta (con `ioctl(I2C_RDWR)`) de 2 mensajes `struct i2c_msg`.
 - Primer mensaje (`Msgs[0]`) `addr = Address; flags = 0;` => esto indica "mensaje de escritura" en la dirección `Address`. Se escriben `OutLen` bytes desde el buffer `Out`.
 - Segundo mensaje (`Msgs[1]`) `addr = Address; flags = I2C_M_RD | I2C_M_NOSTART;` => indica "mensaje de lectura" en la misma dirección, sin soltar la línea (repeated start), leyendo `InLen` bytes hacia el buffer `In`.
- El `ioctl(I2C_RDWR, &Msgset)` combina ambos mensajes en una sola transacción "repeated start" típica para leer registros en dispositivos I2C.
 - Concretamente, el kernel (i2c-dev) hace:
 1. Start, escribe `Address+W`, manda "OutLen" bytes (por ejemplo 1 byte de registro).
 2. Repeated Start, escribe `Address+R`, recibe "InLen" bytes.
 3. Stop.
 - Así se logra el "random read" típico (indicar un registro interno y luego leer).

2. I2C_READ

- Es una versión simplificada donde asume que sólo escribes 1 byte "OutLen = 1" (que a menudo es el registro), y luego lees `Len` bytes.
- Por dentro se llama a `I2C_READ_BYTES` con `OutLen=1`.

3. I2C_WRITE

- Cambia el esclavo con `ioctl(FD, I2C_SLAVE_FORCE, Address)` para indicar con cuál dirección se va a operar.
- Hace un `write()` simple de `Len` bytes.
- Es habitual en muchos dispositivos que la secuencia de escritura sea: [Registro(1 byte) + datos(1 o varios bytes)].

¿Por qué usan `ioctl(I2C_SLAVE_FORCE, Address)` en vez de `ioctl(I2C_SLAVE, Address)`?

- `I2C_SLAVE_FORCE` fuerza el cambio de dirección sin hacer comprobaciones. A veces es un modo más "directo" (potencialmente peligroso si otro proceso está usando el bus), pero en la práctica funciona igual para este caso.

```
#define I2C_READ_BYTES(FD, Address, OutLen, InLen, Out, In, Return) \
{ \
    struct i2c_msg Msgs[2]; \
    struct i2c_rdwr_ioctl_data Msgset[1]; \
    Msgs[0].addr = (Address); \
    Msgs[0].flags = 0; \
    Msgs[0].len = OutLen; \
    Msgs[0].buf = (__u8 *) (Out); \
    Msgs[1].addr = (Address); \
    Msgs[1].flags = (I2C_M_RD | I2C_M_NOSTART); \
    Msgs[1].len = (InLen); \
    Msgs[1].buf = (__u8 *) (In); \
    Msgset[0].msgs = Msgs; \
    Msgset[0].nmsgs = 2; \
    if (ioctl((FD), I2C_RDWR, &Msgset) < 0) { \
        SC_ERR("unable to read from I2C device %x: %m", (Address)); \
        (Return) = -1; \
    } \
} \

#define I2C_READ(FD, Address, Len, Out, In, Return) \
{ \
    I2C_READ_BYTES(FD, Address, 1, Len, Out, In, Return) \
} \

#define I2C_WRITE(FD, Address, Len, Out, Return) \
{ \
    if (ioctl((FD), I2C_SLAVE_FORCE, (Address)) < 0) { \
        SC_ERR("unable to access I2C device %x: %m", (Address)); \
        (Return) = -1; \
    } \
    if ((Return) == 0 && write((FD), (Out), (Len)) != (Len)) { \
        SC_ERR("unable to write to I2C device %x: %m", (Address)); \
        (Return) = -1; \
    } \
} \
}
```

Ejemplo I2C

```
// Quiero leer el valor del registro en offset 0x0 y 2 bytes de respuesta
Out_Buffer[0] = Offset;          // Por ejemplo, 0x0
I2C_READ(FD, IO_Exp->I2C_Address, 2, Out_Buffer, In_Buffer, Ret);
```

Esto:

1. Copia **Offset** (por ejemplo 0x0) en el primer byte de **Out_Buffer**.
2. Llama a **I2C_READ(FD, Address, 2, Out_Buffer, In_Buffer, Ret)**.
3. Internamente, esto hace:
 - Primer i2c_msg => **escribir** 1 byte (el offset 0x0).
 - Segundo i2c_msg => **leer** 2 bytes (los pondrá en **In_Buffer**).

Si no hay error, **In_Buffer** contiene los 2 bytes de datos leídos del offset 0x0.

Ejemplo I2C

Observa que en el archivo `sc_app.c` hay funciones que hacen:

- **Lectura de registros** (por ejemplo `Read_INA226`, `Access_Regulator`, `Access_IO_Exp`, etc.).
- **Escritura de registros** (por ejemplo la parte final de `Access_IO_Exp` con `Op == 1`, o cuando se configura la calibración de un INA226).

Estas funciones siguen un mismo patrón:

1. Abrir el bus I2C con `open(INA226->I2C_Bus, 0_RDWR)`.
2. Llamar a `I2C_READ(...)` o `I2C_WRITE(...)` (que hacen internamente `ioctl()` y `write()/read()`).
3. Cierra el descriptor con `close()`.

Por ejemplo, veamos `Read_INA226(...)` (línea ~1861

```
Out_Buffer[0] = 0x0; // Conf. register(00h)
I2C_READ(FD, INA226->I2C_Address, 2, Out_Buffer, In_Buffer, Ret);
// In_Buffer ahora tiene 2 bytes => Regs->Configuration
Regs->Configuration = ((In_Buffer[0] << 8) | In_Buffer[1]);
```

y así sucesivamente para cada registro (shunt voltage, bus voltage, etc.).

Detalle: para escribir un registro del INA226, la macro es `I2C_WRITE`. Se pone en `Out_Buffer[0]` la dirección de registro, `Out_Buffer[1]` el byte alto, `Out_Buffer[2]` el byte bajo, y luego `I2C_WRITE(FD, Address, 3, Out_Buffer, Ret)`.

Access_Regulator(Voltage_t *Regulator, float *Voltage, int Access)

- Si `Access == 0`, significa “leer el voltaje actual”. Internamente hace un `I2C_READ(...)` para el comando PMBus (por ejemplo `PMBUS_READ_VOUT`).
- Si `Access == 1`, configura primero la página del PMIC (escribiendo un byte con `Page_Select`), y luego hace varios `I2C_WRITE(...)` para setear valores (ej. `VOUT_COMMAND`, `OV_WARN_LIMIT`, etc.).
- Cada vez, llama a las macros como `I2C_WRITE(...)`, `I2C_READ(...)`.

Access_IO_Exp(IO_Exp_t *IO_Exp, int Op, int Offset, unsigned int *Data)

- Si `Op == 0`, es lectura:

```
Out_Buffer[0] = Offset;
I2C_READ(FD, IO_Exp->I2C_Address, 2, Out_Buffer, In_Buffer, Ret);
*Data = ((In_Buffer[0] << 8) | In_Buffer[1]);
```

Si `Op == 1`, es escritura:

```
Out_Buffer[0] = Offset;
Out_Buffer[1] = ((*Data >> 8) & 0xFF);
Out_Buffer[2] = (*Data & 0xFF);
I2C_WRITE(FD, IO_Exp->I2C_Address, 3, Out_Buffer, Ret);
```

EEPROM_IDT_8A34001(...) y Set_IDT_8A34001(...)

- Configuran un chip de reloj IDT 8A34001 escribiendo secuencias de datos (a menudo piden data files tipo TCS, BIN, etc.). Igualmente usan `I2C_WRITE(...)` e `I2C_READ(...)` para mandar/leer bytes.

Pasos a seguir

¿Que haria yo?

1. **Identificar las funcionalidades básicas que quiero que tenga mi libreria**
 - a. Por el momento me centraría en poder leer algún que otro sensor
2. **Copia (o crea) el archivo JSON en tu proyecto**
 - a. Básicamente nos da toda la información que queremos sobre la tarjeta(VCK 190)
 - b. Está pensado para el SC, así que hay que verificar si tanto las direcciones de los sensores como el número de bus están bien o no (mirar footprints)
3. **Define estructuras de datos en C (o C++) similares a las de sc_app.h**
 - a. Observa cómo en sc_app.h está la estructura Plat_Devs_t, que contiene punteros a Clocks_t, INA226s_t, Voltages_t, etc.
 - b. Crea las mismas struct en tu nuevo proyecto (puede ser un archivo my_sc_app.h)
4. **Implementa el parseo del JSON (similar a Parse_JSON(...))**
 - a. La función Parse_JSON(const char *File, Plat_Devs_t *Plat_Devs) abriría el JSON, buscaría las secciones ("INA226", "CLOCK", etc.) y rellenará las estructuras.
 - b. Al final, tendrás una variable global o estática Plat_Devs_t *Plat_Devs con los datos del JSON cargados en memoria.
5. **Crear funciones básicas de I2C y lectura de sensores**
 - a. puedes copiar la estrategia de macros (I2C_READ, I2C_WRITE) con ioctl(I2C_RDWR, &msgs) que se ve en sc_app.h.
 - b. Si estás en Linux, abrirías el bus con:
 - c. Haz primero una versión sencilla (sin macros complicados) para verificar que puedes leer un registro. Luego, si quieres, migrar al estilo macros.
6. **Lectura de un sensor INA226 (lo más sencillo)**
 - a. Abre el bus I2C que te indique la estructura
 - b. Lee un registro, por ejemplo la calibración (0x05).
 - c. Cierra el bus.
7. **Implementa una función (parecida a Read_INA226) que devuelva, por ejemplo, el busVoltage o la corriente. Algo como:**
 - a. Con esto, ya tienes la lectura de un sensor. (mirar las de la librería)
8. **Amplía para más sensores**

```
int fd = open(INA226->I2C_Bus, O_RDWR);
ioctl(fd, I2C_SLAVE_FORCE, INA226->I2C_Address);
```

```
int fd = open("/dev/i2c-4", O_RDWR);
ioctl(fd, I2C_SLAVE_FORCE, devAddr);
...
close(fd);
```

```
float readINA226BusVoltage(INA226_t *sensor) {
    int fd = open(sensor->I2C_Bus, O_RDWR);
    if (fd < 0) { ... error ... }

    ioctl(fd, I2C_SLAVE_FORCE, sensor->I2C_Address);

    // Escribo 1 byte: 0x02 (reg de bus voltage)
    uint8_t outbuf[1] = { 0x02 };
    if (write(fd, outbuf, 1) != 1) { ... error ... }

    // Leo 2 bytes
    uint8_t inbuf[2] = {0};
    if (read(fd, inbuf, 2) != 2) { ... error ... }

    close(fd);

    uint16_t raw = (inbuf[0] << 8) | inbuf[1];
    // Convertir segun la datasheet: 1.25 mV por bit
    float voltage = (raw * 1.25f) / 1000.0f;
    return voltage;
}
```

!!! Muy importante, de momento solo leer registros, nada de escribir, podríamos cargarnos la tarjeta !!!

- Integrar la lectura con el parseo JSON

Una vez que tengas:

1. `Plat_Devs_t` con `INA226s_t` parseado del JSON.
2. Funciones `readINA226...` que esperan la dirección/bus del sensor.
3. Entonces, para leer **todos** los INA226 enumerados en tu JSON, podrías hacer algo como:
4. Y listo: tienes un “mini system controller” que parsea la configuración y lee los sensores.

```
void readAllINA226(Plat_Devs_t *plt) {  
    for (int i = 0; i < plt->INA226s->Numbers; i++) {  
        INA226_t *dev = &plt->INA226s->INA226[i];  
        float v = readINA226BusVoltage(dev);  
        printf("Sensor %s -> %f V\n", dev->Name, v);  
    }  
}
```

Agregar más dificultad / funcionalidades paso a paso

A partir de aquí, vas sumando:

1. **GPIO** (como hace el original con `gpiod_ctxless_find_line` y `Get_GPIO`).
2. **PMBus** para regular voltajes (similar a `Access_Regulator`).
3. **Clocks** (lectura/escritura Si570, IDT...), si te interesa manipular frecuencias en la APU.
4. **IO Expander** (lectura y escritura).
5. **Memoria EEPROM** (igual a `geteeprom`, `EEPROM_Common`, etc.).
6. **Tests BIT** (descargar un tcl al XSDB, etc.).

Cada bloque se añade copiando la lógica del `sc_app.c`, usando tus propias funciones “i2c_xxx” y “gpio_xxx” en la APU.

Recomendaciones adicionales

1. Mantener la misma organización

- Un archivo (por ejemplo, `my_sc_parse.c`) para parsear el JSON.
- Uno para la lógica de “INA226”, otro para “voltages”, etc.
- Una cabecera “`my_sc_app.h`” con las struct.

2. Ir probando en hardware real

- Con un único sensor (INA226) y un bus I2C (`/dev/i2c-4`). Imprime la lectura por consola.
- Valida la dirección I2C y que te dé valores coherentes.

3. Incorporar logs o debugging

- Igual que el original, puedes usar macros `SC_ERR(...)`, `SC_INFO(...)`.

4. Replica gradualmente

- No intentes portar todo a la vez. Comienza con “INA226 + parseo JSON” y, cuando funcione, añade “GPIO + parseo JSON”, etc.

EXTRA

Localizar sensores y direcciones

Ver buses i2c: Como vimos antes, al activar los sensores desde la configuración del PMC en vivo, se activan los buses i2c-1 y 2. Para leerlos:

i2cdetect -y 0

```
[root@fanae51 ~]# i2cdetect -y 0
Warning: Can't use SMBus Quick Write command, will skip some addresses
 00:  0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
10:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
```

suggests that **some I2C devices do not support the SMBus "Quick Write" command**. This typically happens because:

1. **The devices on the bus are standard I2C (not SMBus)**
 - o The `i2cdetect` tool **by default** tries SMBus "Quick Write" commands, but many I2C peripherals **do not support it**.
 - o Examples: I2C **multiplexers** (TCA9548APWR) and **PMBus power monitors** often reject SMBus-specific commands.
2. **I2C bus drivers are not fully initialized**
 - o If the **kernel does not properly support SMBus transactions**, the I2C controller will reject these commands.
 - o This is sometimes an issue with **ZynqMP/Versal I2C drivers**.
3. **Devices behind the I2C multiplexers are not selected**
 - o You see **0x74 and 0x75**, which are **TCA9548A I2C multiplexers**.
 - o These multiplexers **hide other devices** behind different channels.
 - o If no channel is selected, **the downstream devices will not be visible**.

Fix: Use -r (Raw Mode) and Select an I2C Mux Channel

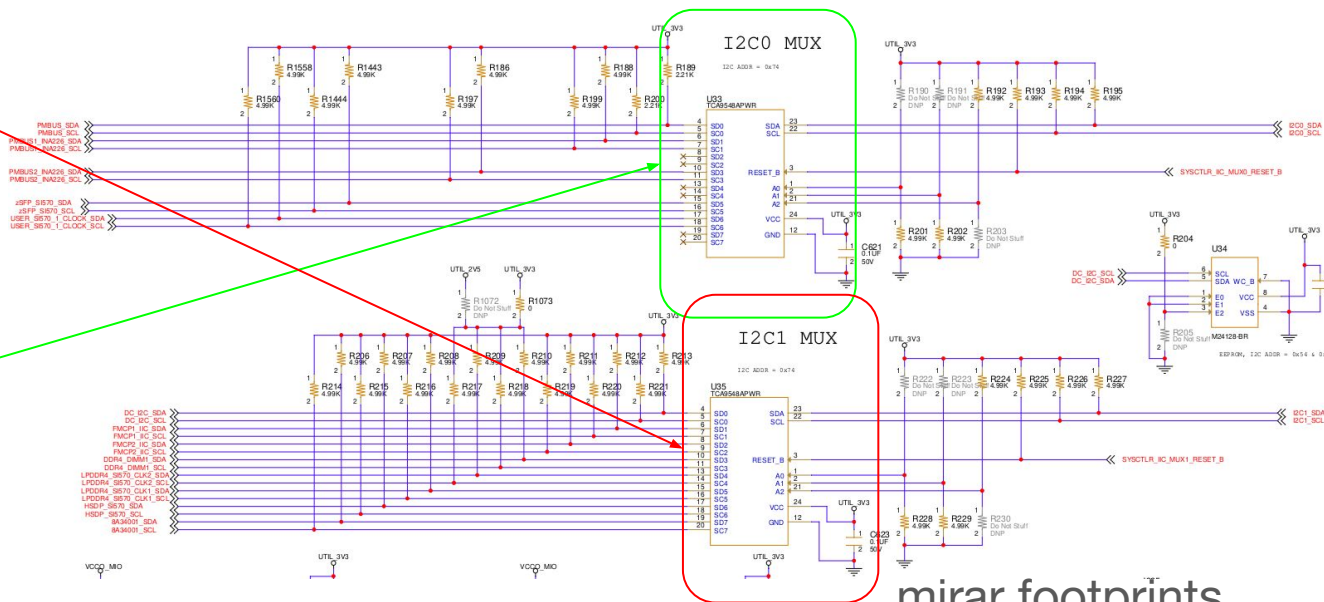
i2cdetect -y -r 0

```
[root@fanae51 ~]# i2cdetect -y -r 0
 00:  0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
10:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70:  -- -- -- -- 74 75 -- -- -- -- -- -- -- -- --
```

```

y
[root@fanae51 ~]# i2cdetect -y -r 2
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
10:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
20: 20  --  --  --  --  --  --  --  --  --  --  --  --  --  --
30:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
40:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
50:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
60:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
70:  --  --  --  -- 74  --  --  --  --  --  --  --  --  --  --

```

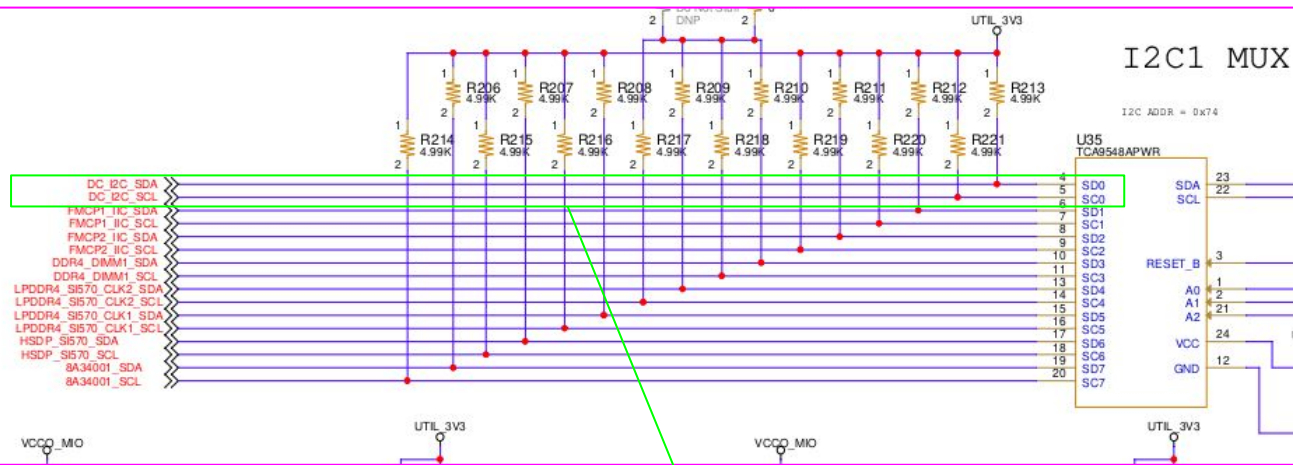


mirar footprints
página 44

I2C Multiplexer (TCA9548APWR) Details

- These multiplexers are responsible for switching different I2C buses to access multiple sensors, power monitors, and other peripherals.
- **I2C0 and I2C1 are each connected to a separate TCA9548APWR.**
- Each multiplexer can control **up to 8 different I2C channels**, allowing access to various onboard sensors and components.

https://www.hep.uniovi.es/pleguina/carolina/versal_monitor+manager+info.zip



I2C1 MUX

I2C ADDR = 0x74

Veo que está conectado a la memoria EEPROM de la tarjeta.

```
i2cset -y 0 0x74 0x01 # Enable channel 0 on I2C0 Mux
```

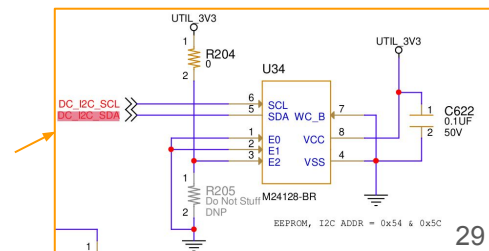
Activo el canal 1 del multiplexor para tener acceso a esas direcciones

```
[root@fanae51 ~]# i2cset -y 0 0x74 0x01 # Enable channel 0
[root@fanae51 ~]# i2cdetect -y -r 0
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
10:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
20:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
30:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
40:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
50:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
60:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
70:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
```

Ahora que he activado el canal, tengo acceso a la memoria. Esto funciona así para todos los periféricos.

Para ver a qué dispositivos está conectado cada multiplexor solo tengo que seguir las señales

Por ejemplo el I2C mux (i2c_2 en linux), en el canal 0, está conectado a la señal DC_I2C_SDA/SCL, busco en las footprints ese nombre para ver a donde me lleva...:



```

[root@fanae51 ~]# i2cset -y 2 0x74 0x01 # Enable channel 0
[root@fanae51 ~]# i2cdetect -y -r 2
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
10:  --  --  --  --  --  --  17  --  --  --  --  1c  1d  1e  --  --
20:  20  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
30:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
40:  --  --  --  --  --  46  47  --  --  --  --  --  4c  4d  4e  4f
50:  50  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
60:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
70:  --  --  --  --  74  --  --  --  --  --  --  --  --  --  --  --

```

Por ejemplo, voy a activar el canal 0 del i2c bus 2 , que en el esquemático nos dice que es PMBUS.

PMBus (Power Management Bus) es un protocolo basado en I2C, diseñado específicamente para la gestión de energía. Se usa en reguladores de voltaje, monitores de energía y fuentes de alimentación para controlar y supervisar el consumo de energía en dispositivos electrónicos.

Información del dispositivo

Comando	Descripción	Ejemplo en Linux
0xAD	Leer ID del fabricante	<code>i2cget -y 0 <dirección> 0xAD</code>
0xD4	Leer tipo de dispositivo	<code>i2cget -y 0 <dirección> 0xD4 w</code>
0xD5	Leer versión de firmware	<code>i2cget -y 0 <dirección> 0xD5 w</code>

⚡ Comandos de voltaje

Comando	Descripción	Ejemplo en Linux
0x8B	Leer voltaje de salida	<code>i2cget -y 0 <dirección> 0x8B w</code>
0x20	Leer voltaje nominal	<code>i2cget -y 0 <dirección> 0x20 w</code>
0x21	Leer voltaje mínimo	<code>i2cget -y 0 <dirección> 0x21 w</code>
0x22	Leer voltaje máximo	<code>i2cget -y 0 <dirección> 0x22 w</code>

🔌 Comandos de corriente y potencia

Comando	Descripción	Ejemplo en Linux
0x8C	Leer corriente de salida	<code>i2cget -y 0 <dirección> 0x8C w</code>
0x96	Leer potencia de salida	<code>i2cget -y 0 <dirección> 0x96 w</code>
0x97	Leer potencia máxima de salida	<code>i2cget -y 0 <dirección> 0x97 w</code>

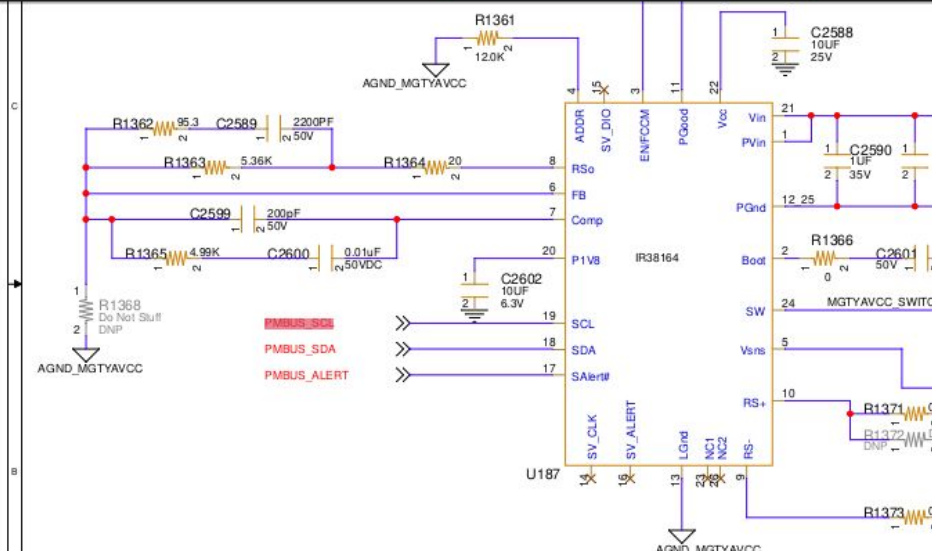
🔧 Comandos de estado y fallos

Comando	Descripción	Ejemplo en Linux
0x78	Leer estado general	<code>i2cget -y 0 <dirección> 0x78 w</code>
0x79	Leer estado de fallos	<code>i2cget -y 0 <dirección> 0x79 w</code>
0x80	Leer fallos de voltaje de entrada	<code>i2cget -y 0 <dirección> 0x80 w</code>

Ejemplo: **Leer el voltaje de salida** de un dispositivo PMBus en **0x40**:

```
i2cget -y 0 0x40 0x8B w
```

SDA PMBUS_... 44
 3V3 PMBUS_S... 44
 10 PMBUS_SC... 57
 17 PMBUS_SC... 61
 17 PMBUS_SC... 63
 17 PMBUS_SC... 65
 9 PMBUS_SCL... 69
RSPMBUS_SC... 70
 NC2 PMBUS_... 71



Si yo busco
 PMBUS_SCL me
 salen todos los
 dispositivos
 conectados al bus.

Voy a buscar el 0x1c

Al revisar el **footprint del regulador** en el diseño del **VCK190**, ves que tiene **dos direcciones I2C diferentes**:

- **0x1C** → Dirección estándar I2C
- **0x4C** → Dirección PMBus (Power Management Bus)

Esto ocurre porque algunos **reguladores de voltaje avanzados** (como **Intersil**, **Texas Instruments**, **Maxim**, etc.) soportan **ambos protocolos**:

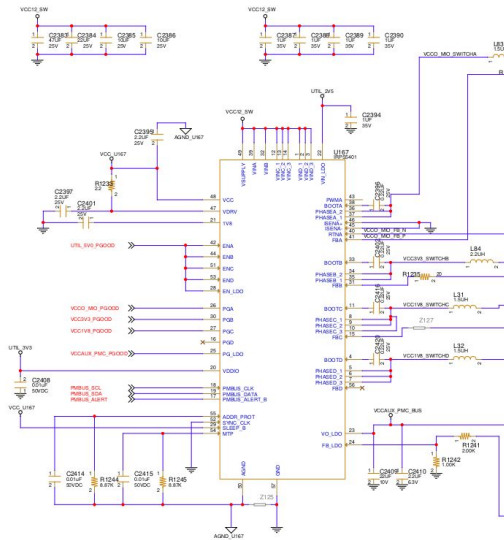
1. **Modo I2C genérico** (dirección 0x1C)
 - Permite acceso a **configuración básica del dispositivo**.
 - A veces solo permite comandos de **identificación**.
2. **Modo PMBus (0x4C)**
 - Implementa comandos más avanzados como:
 - Medición de **voltaje**
 - Medición de **corriente**
 - Ajuste de **voltaje de salida**
 - Monitoreo de **fallos y estado**

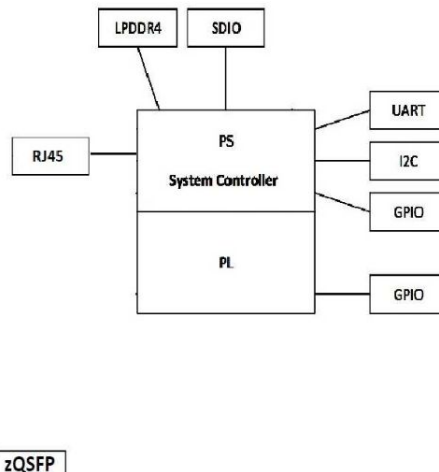
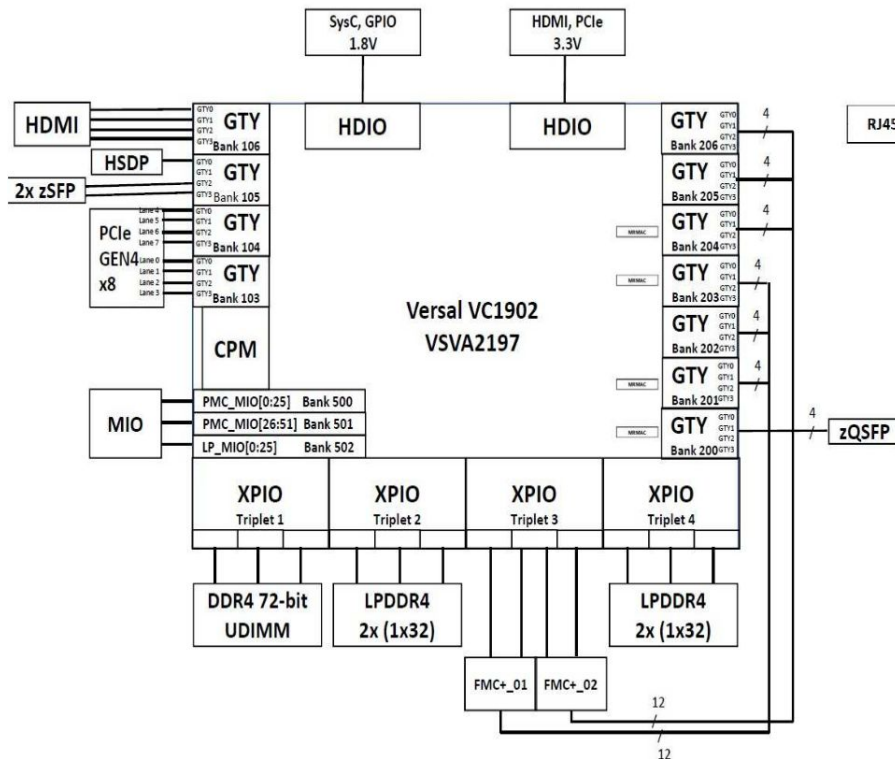
Si aparecen **ambas direcciones en la misma línea de I2C**, significa que el **dispositivo responde en ambos modos**.

MTP: +C
7-BIT I2C ADDR: **0x1C**
7-BIT PMBUS ADDR: **0x4C**

```
[root@fanae51 ~]# i2cget -y 2 0x1C 0x00
0x00
[root@fanae51 ~]# i2cget -y 2 0x4C 0xAD
0x01
[root@fanae51 ~]# i2cget -y 2 0x4C 0x8B w
0x0300
[root@fanae51 ~]# i2cget -y 2 0x4C 0x8C w
0xb05a
```

Comando	Dirección	Resultado	Interpretación
i2cget -y 2 0x1C 0x00	0x1C (I2C)	0x00	No responde con datos útiles (acceso I2C básico).
i2cget -y 2 0x4C 0xAD	0x4C (PMBus)	0x01	Identificación del dispositivo (Fabricante o ID de tipo de dispositivo).
i2cget -y 2 0x4C 0x8B w	0x4C (PMBus)	0x0300	Voltaje de salida (RAW, necesita conversión).
i2cget -y 2 0x4C 0x8C w	0x4C (PMBus)	0xb05a	Corriente de salida (RAW, necesita conversión).





1. Identificación de los periféricos accesibles

Antes de escribir código, hay que determinar qué sensores o dispositivos están accesibles en la placa.

1 Revisar el hardware en la hoja de datos de la VCK190

- Descarga el **User Guide** de la VCK190 y busca la sección de **periféricos accesibles**.
- Verifica qué sensores están conectados a **I²C**, **SPI** o **GPIO**.
- Busca en el **schematic** de la placa si los sensores tienen una conexión fija a la **PS (Procesador ARM)** o **PL (FPGA fabric)**.

2 Consultar los footprints del PCB

- Revisa el **footprint del PCB** para verificar la disponibilidad física de los periféricos.
- Identifica los **pinos PC disponibles** y a qué dispositivos están conectados.

3 Determinar direcciones PC de los dispositivos

- En la documentación de la VCK190, busca las direcciones **PC de cada sensor** conectado al bus.
- Si no están documentadas, se pueden escanear con un **PC bus scanner** desde Linux o Baremetal.
- También pueden buscarse en los footprints